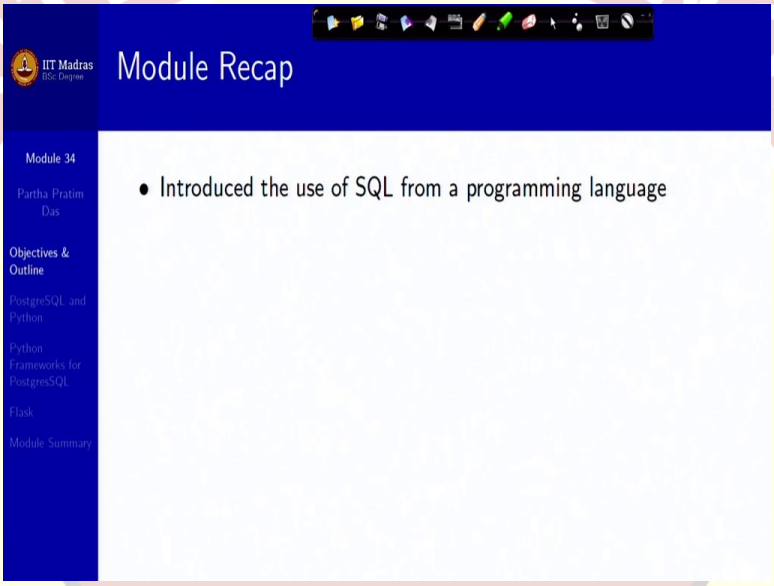
IIT Madras

ONLINE DEGREE

Database Management System
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Application Design and Development/4: Python and PostgreSQL

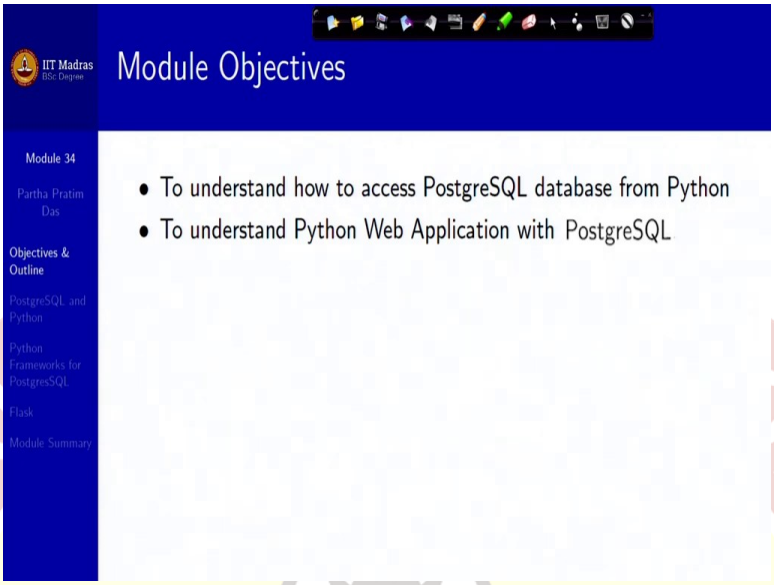
Welcome to Module 34 of Database Management Systems course, in the IIT Madras online B.Sc. Program.

(Refer Slide Time 0:28)



We have been talking about application, design and development, from the beginning of this week, and we have discussed about the application architecture, we have talked about fundamentals of the web in case, you are not aware of that, and we specifically, in the last module, talked about how to use SQL from a programming language like C, C++, Java, Python and so on. We have seen the connectionist as well as the embedded mechanisms for doing that.

(Refer Slide Time 1:04)

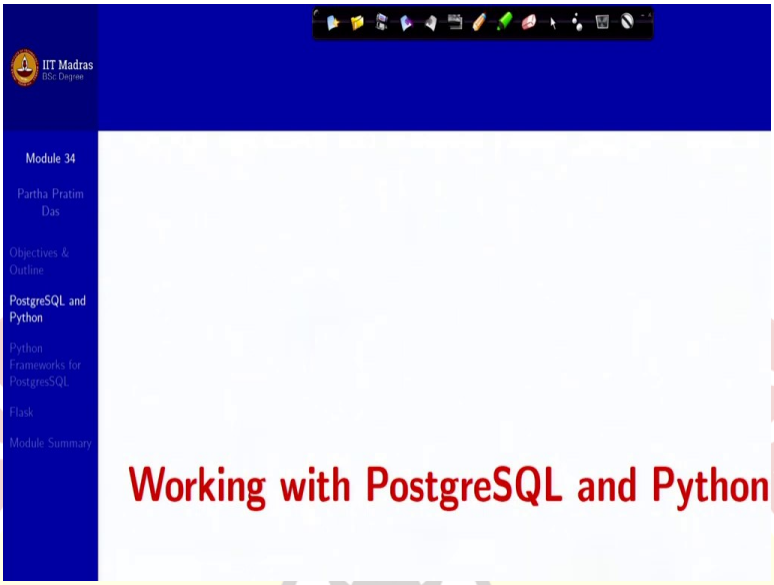


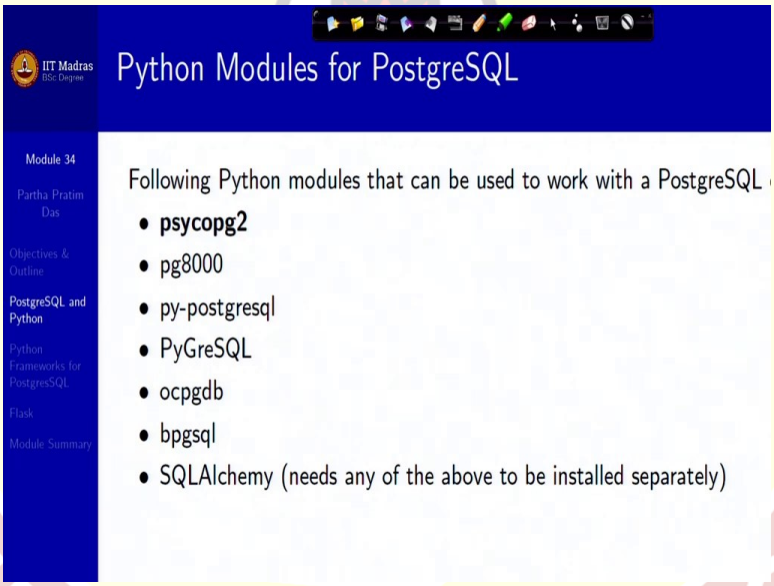
In the module today, we will take this forward with a more specific example, which is kind of on one side, deals with most popular choice, I would say, of language for development, which is Python, and as an example, I am using PostgreSQL so that you can easily try out because it is, it is, free open source application.

So now, we will specifically focus on a complete application development, whether be it in the front end GUI as well as in terms of the back-end data access. Of course, not getting into the middle tier because that has, that has to have all the different business logic and it is a matter of more and more simple programming and abstraction into those programming snippets into different functions which you can always do.

So, we would like to show the two extremes, one is how do you get into the front end and how to easily do that and how to access the data, update the data. And we will also discuss about the fact that besides direct library-based provisions with Python for PostgreSQL, there are several frameworks also, which allow Python to be used with different databases. So that is our scope today.

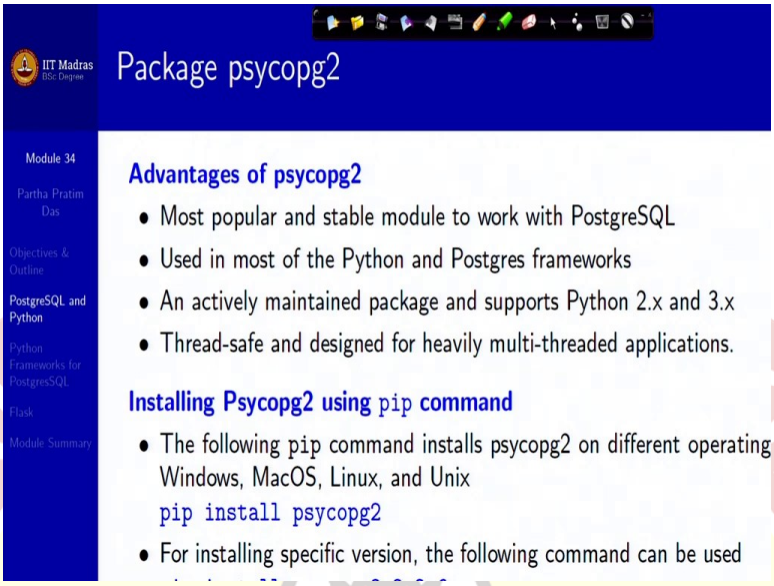
(Refer Slide Time 2:47)





So, working with PostgreSQL and Python. Now, Python have several modules that can be used to work with a PostgreSQL data server, database server. And just to name a few, we would focus on the first one. They are very similar. If you have seen the functionality, they are very similar, but there are differences, of course. But we will focus on the, the first one, psycopg2, because it is one of the most popular in the community right now.

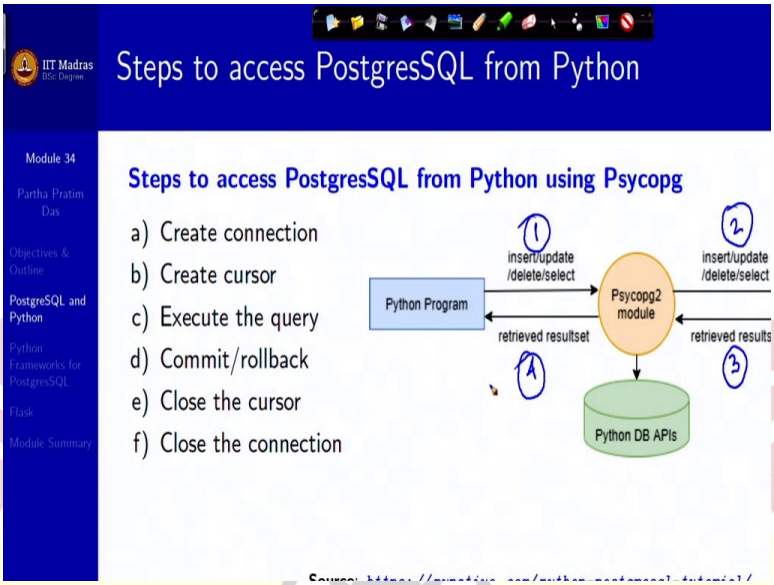
(Refer Slide Time 3:24)



So, it is stable, in terms of PostgreSQL and used most of the Python and Postgres frameworks as well. It is an, it is, because, Python is community managed, community developed, so it depends on which is popular in community so that the community members are supporting it, maintaining it. So, this particular package is actively maintained, supports both Python 2.x and 3.x, it is thread safe. And particularly, designed for heavily multi-threaded applications which makes it particularly suitable for our use here.

So these are the, using the pip command you can install this module, this package onto your system. You can try that out.

(Refer Slide Time 4:19)



Now, going back to the steps, to access the PostgreSQL from Python, using this module, we are primarily focusing on the back end side. Then we will come on the GUI front-end side later. So, it is, it follows almost the same workflow, as we had discussed in terms of the other programming languages connecting to, using SQL, that you need to create a connection that is, first important, otherwise you cannot do anything with the database.

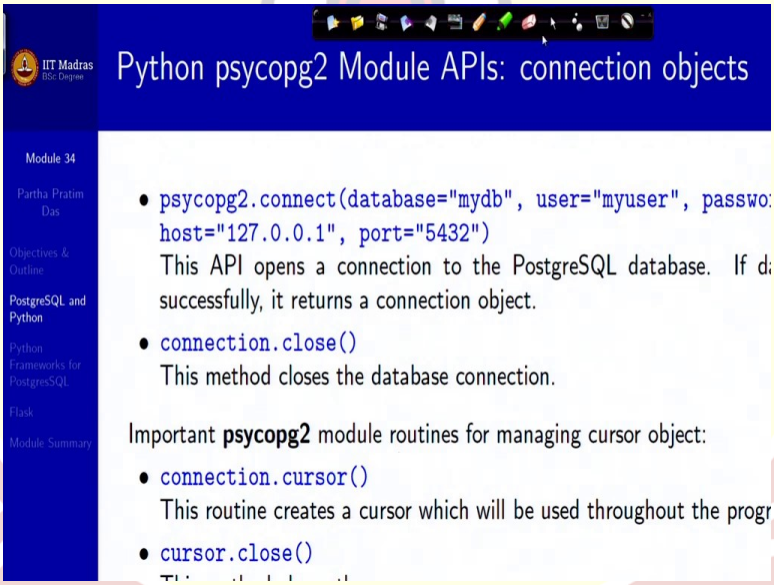
Then you need to create a cursor. Cursor as you remember, is a iterator, to the result set that I expect to get so that I can actually present that in my programming language. We need to be able to execute the SQL query and here, what we have added is, there is a direct provision to commit or rollback. That is, depending on the programming logic, you could decide that after a query has been executed the changes, whether they should be committed or you want to rollback to the state where you were before these changes had happened.

We will talk about rollback situations a lot, much later. But you can just conceptually take it that when you start doing a transaction, that is, you are running some Python function which is making changes to the database through the execute query, if you are not, if you think that something is wrong, something, some logic did not, was not correct, some input was not proper, so you might want to rollback these changes and come to the state before you had called this task function. Or if you are happy, then you can commit.

Once, that is done, it is time to close the things, so you close the cursor, release it and finally close the connection. A very simple workflow and a very intuitive one. So, if you see from here, there are three components. One is a Python program, which is your middle tier language, accessing the database. Here is your database and sitting in between is the package. So, it is a module and the Python database APIs that we are going to take a glimpse of.

So, initially, the Python program sends a request for, sets a request for insert, delete, update or select, using the method calls on the module. That in turn will create, will initiate an insert update delete select as it is, using SQL on the database. After this is done, if there is a result set to be returned, that is retrieved. From the database, by the module. And finally, that is given back as a result set, with the cursor to the Python program. So, very simple steps. We have to just see what are these specific calls going to be.

(Refer Slide Time 7:43)

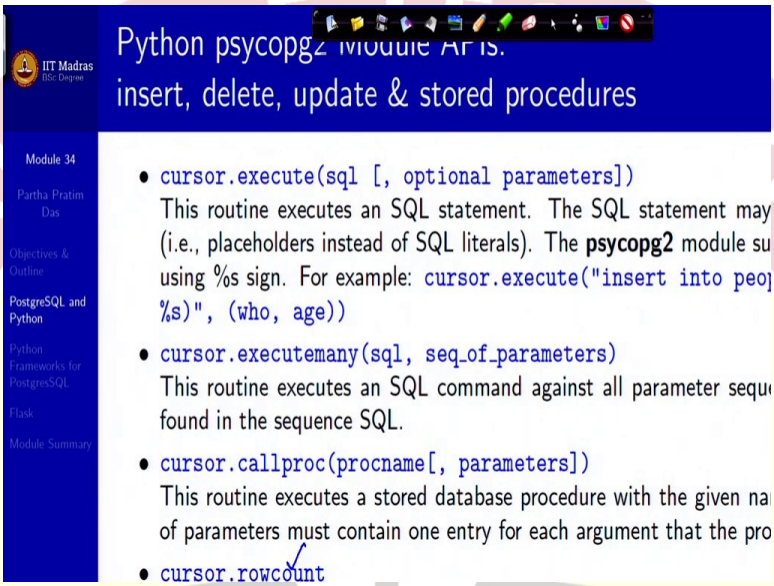


So, first is connect, which is very similar to the connect we saw earlier. It is called connect. And since, it is, it is kind of a static function, you are starting at this point, so it has to be called on the module. You specify the database, the user, the password, the host, the port, all details, I mean, where the database server is existing, what is the name of the database and who wants to connect, and then it will open a connection.

It will open a connection, it will return a connection object. Now, if you want to close that connection, you call, invoke the method close on the connection object. So that is what you will

do at the end. Otherwise the next step is, you need a cursor, so you do connection dot cursor which is where, connection is your connection object, you have got the current connection. It will create a cursor which can be used throughout the program. And when you are done, you close and release the cursor. Very simple. So, beginning and end, we have done here.

(Refer Slide Time 8:53)



Then, rest of the stuff mostly you do with the cursor because that is your handle to the, to the connection and the operation, so, on the cursor, you execute, you do the method, execute. And, the, this will execute an SQL statement, which may be parameterized by several placeholder.

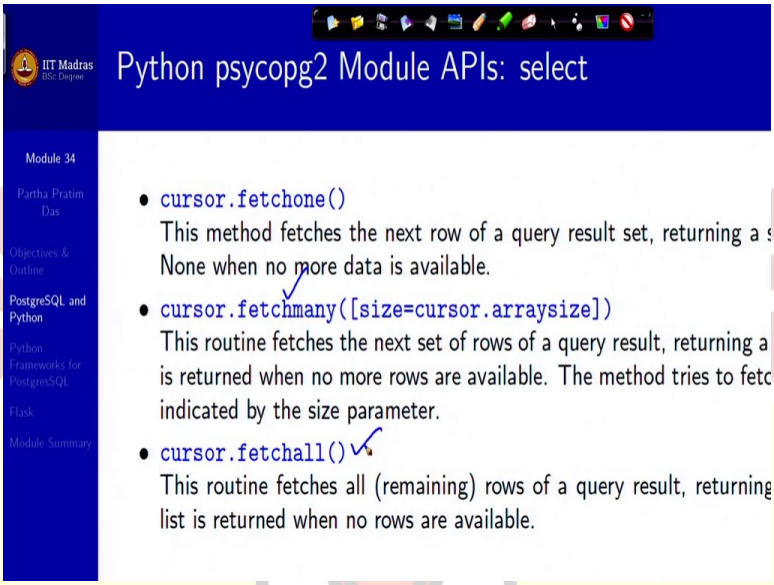
So, it supports sees print s, kind of, so, these are examples where this is your string, SQL command string and your parameters are insert these 2 values, you have put as percentage and you can pass these values specifically as the optional parameters as in given here. So, this is a general structure of the SQL execute.

You can actually execute number of SQL commands against all parameters sequence or mapping that are found in the sequence. So, you can do a bundle of SQL sequence also. Or you can call a stored procedure. Stored procedure again is something which we have not done in depth, in, these are procedures written in SQL, which are stored in the database itself, pre-compiled, and they may have parameters. So, these stored procedures can be called from the cursor.

You can get the read count, row count, I am sorry, which is a read-only attribute, which returns the total number of database rows that have been modified, inserted or deleted. So, if I do an

execute, certain number of rows will get changed. So, that count will be returned in this read-only member of cursor. Very simple.

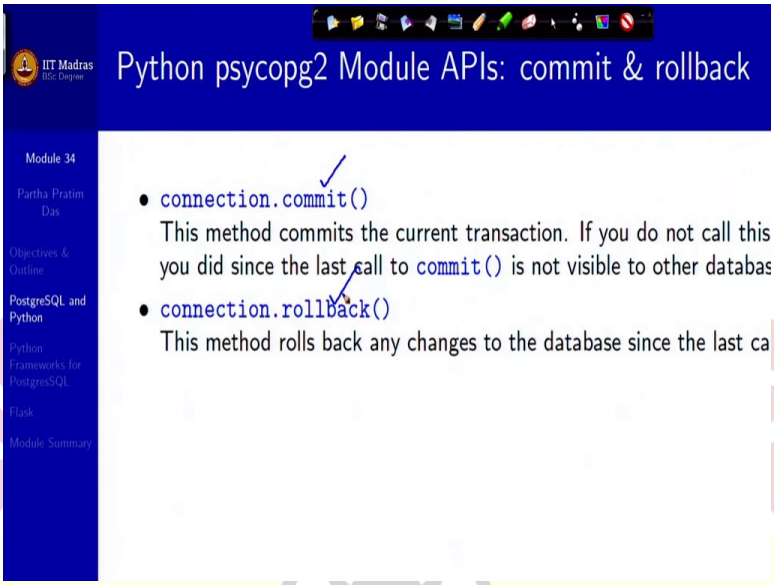
(Refer Slide Time 10:49)



So, then the cursor can actually fetch the result set. It can do fetch one, when it gets the next row of the result set, returning a single sequence, and it will not return if it has got no more data is available. So, it will return in row which does not exist.

And otherwise, you can actually fetch many also. So, if you fetch many rows, then naturally, you will have to put them in an array so you have to set the size of that array as a cursor array size that is, as many, as is the size of the cursor, you will have to get whole of that. And if no rows are available, an empty list will be returned. You can do fetch all to return everything, the remaining also. These are different ways you can do fetch.

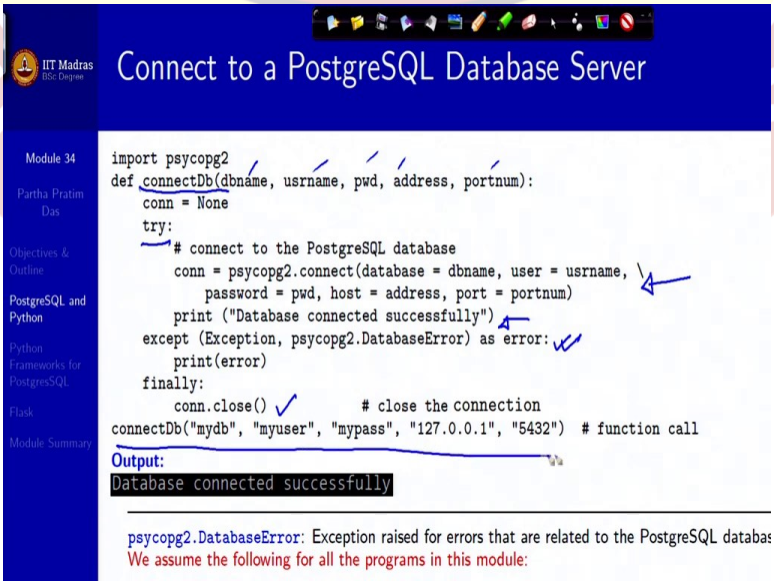
(Refer Slide Time 11:49)



So, that basically gives us the overall connection, execution and closure. The remaining things are as I said, is commit, so this method commits the current transaction. So, once you call that, the changes made in the database are, will become permanent. If you do not call this change, if you do not call commit, then any change that you have made are not visible to other database connections on the same database.

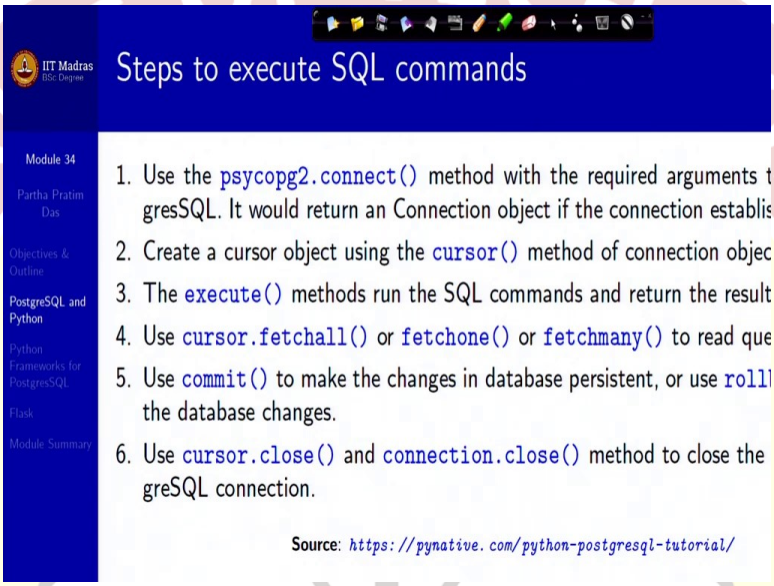
Or you can rollback any change that you have done since the last commit. So, a very simple model to use and here are some examples, quickly.

(Refer Slide Time 12:33)



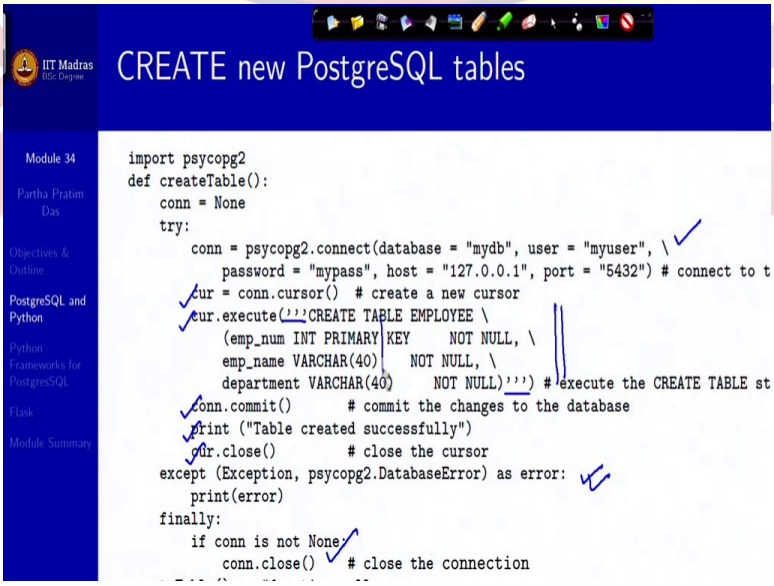
So, we are defining a Python method, connect DB. It has, you can make out, all the required parameters. This is where you are doing the connect and then you say it is connected, it is put within a tri-block. So, if it does not connect, it throws an exception and you will be here with the error and you print that error. Otherwise, and when you are done, you will close the connection. So this is, so, this is the function call, typical function call.

(Refer Slide Time 13:18)



Now, steps to execute the SQL command, we have already understood. Connect, make the cursor, execute, fetch results, commit, close cursor, close connection.

(Refer Slide Time 13:31)

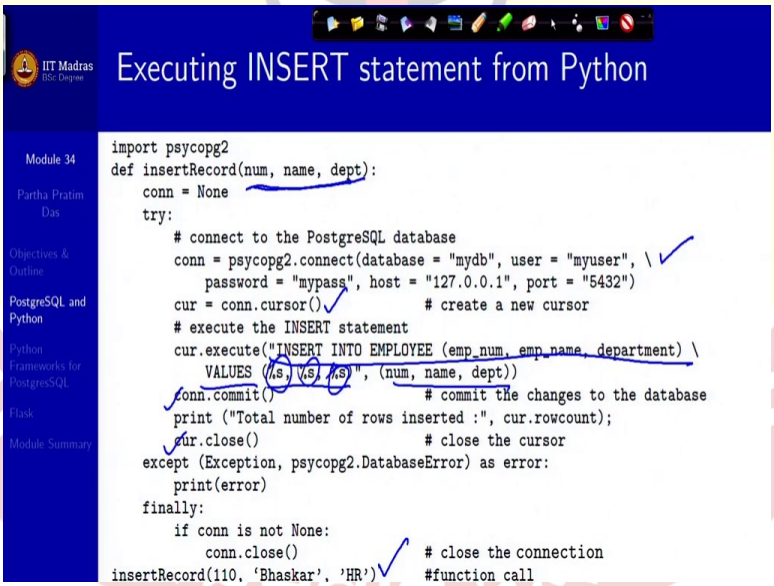


So, we are going to do that now. So this is, this is the connect. If it does not work, it comes to the exception. So, you do not have to check. It is exception based, which is, makes it clean. Then, you have the cursor, which you open on that connection. On the cursor, you execute it, create table. So, you are sending a create table command to be executed.

A table will be created with all this name and, it is exactly a SQL command. It is only that since it is long, it is written as a multi-line string so you have the multi-line string markers of Python. So, this will execute a create table. If it cannot, it will throw an exception, it will be here. So, that is, that is the easy part. You do not have to take care of, if something does not work what will happen. It will always come to the exception and take care of it, and you will be able to close.

Once execution is done, you commit the changes to the database, and you print that the table is successfully created. You close the cursor and finally if connection was not done, then you can close the connection. So, create table call will do this entire thing. You could have passed many of these as parameters, which we have not done here.

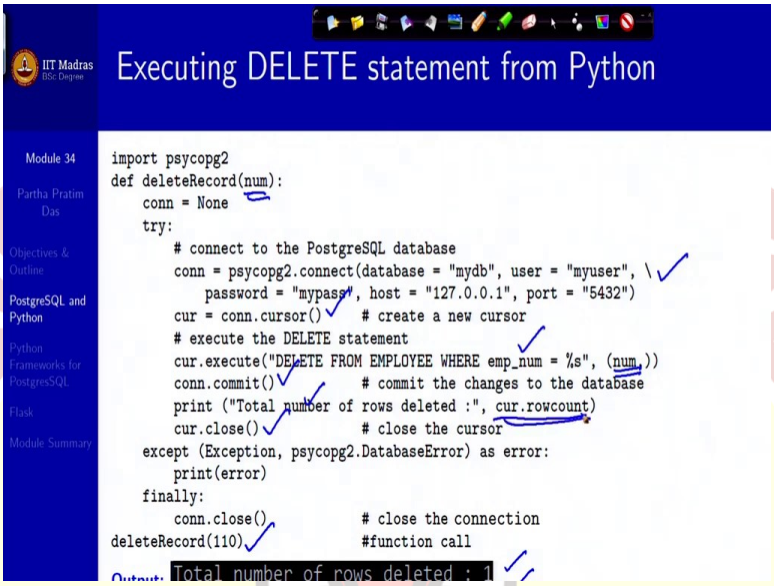
(Refer Slide Time 14:58)



There is a insert example. So, connect, cursor, now you are putting an insert SQL command. You are using three parameters, with the values that you have got from the parameters of this method. So, you can parameterize, as I was saying. So, this is how you can parameterize. These, we will get from, whatever you get from here, will be actually put here. And then you commit, print,

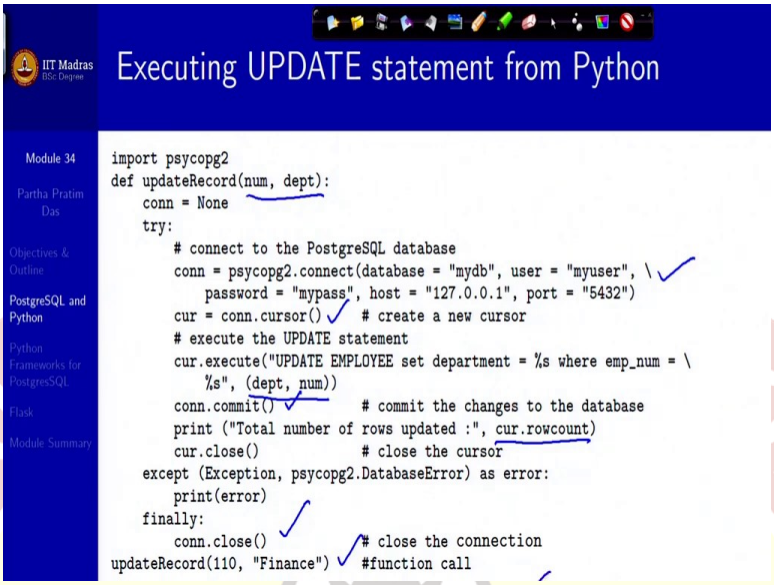
closed, rest of it is, so you can, this is how you invoke. As you do, you see that one record has been added.

(Refer Slide Time 15:41)



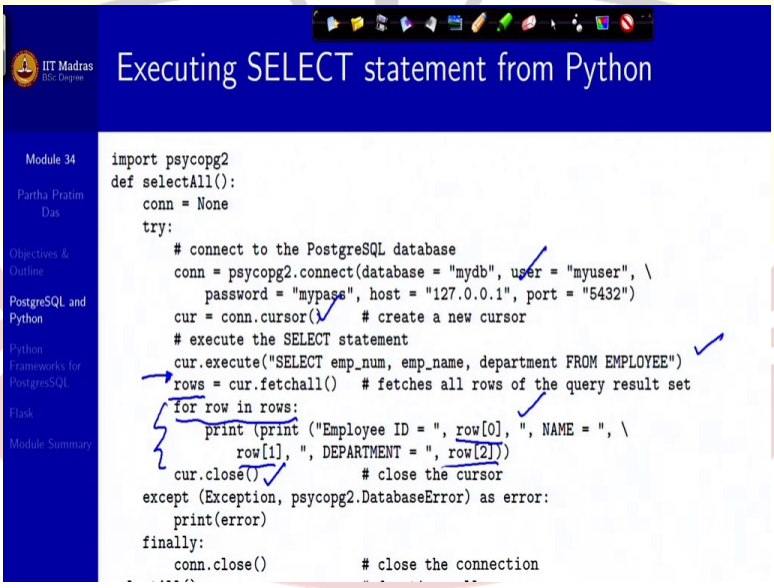
Delete works exactly similarly. Connect, get cursor, execute the number, is a parameter to the method which you set in the delete command. Then you commit, print, close and this is how you actually do a delete and it will, this is the output you will get, that one row has been deleted. If no rows are there to delete, or if it does not, if it cannot delete based on the condition that you have given, then it will print the total number of rows deleted is 0. So, you can see here that we are using the row count member of the cursor for doing this.

(Refer Slide Time 16:33)



UPDATE, this is, this is, I mean, you have got the pattern. It is the same. Connect, cursor, update, giving the parameters, commit, print, how many updated and close. This is how you use, here is one update that is happening.

(Refer Slide Time 16:57)



So if you just, I mean, all these codes are given here. You can just copy paste here or you can actually use the source that we have given. Finally, the select, so you have connect, you have cursor, you do a select, here we are doing select all. So, we have not passed any parameter. Now, once, you have done this, then your cursor has the result set. So, you are, getting all that into a

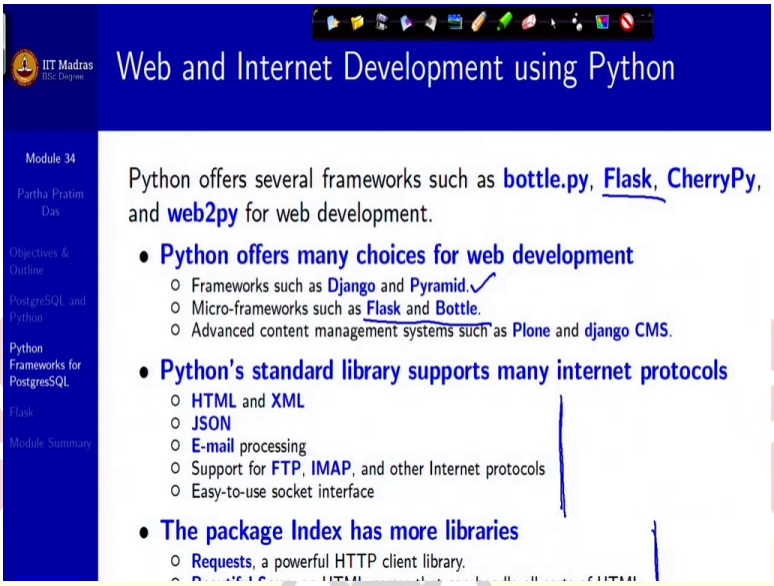
list. You said rows is assigned, cursor dot fetch all. So, everything that you have got is put in here and then you run a iterator on the rows.

So, you go over rows till there are rows and you keep on printing the current values, these different components, row 0, row 1, row 2, at the free components of the record and you keep on doing this on the list till the list does not get exhausted. There is no question of commit, because you have not made any changes. And, so, once you are done, you simply close the cursor, close connection and here, this is a typical result that you might get.

(Refer Slide Time 18:13)

So, that was how you can very easily use Python with a database server like PostgresSQL using the, I mean, there are other modules for other database as well. You can use this module and very easily do this. I think all of you would be able to write the programs for this.

(Refer Slide Time 18:36)



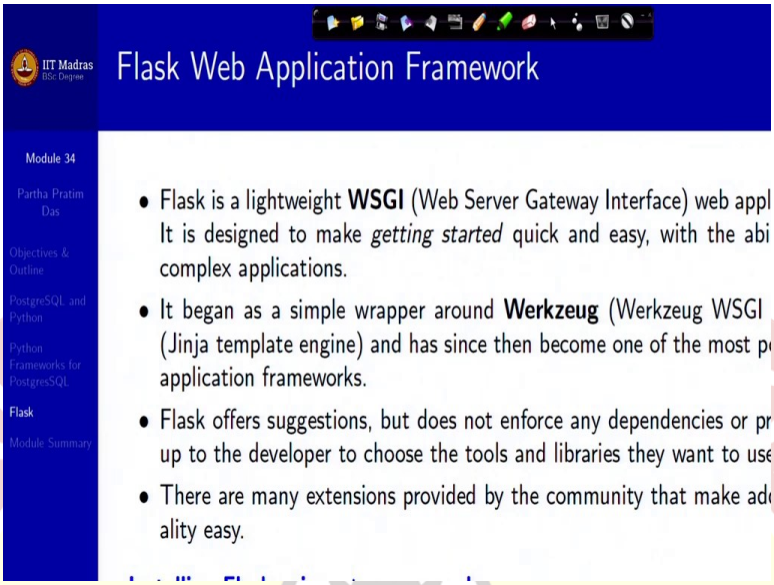
Now, let me go into something more, I mean, more of convenience, which says that there are different web and internet development frameworks. It is not only just a module, but a framework which has, besides such a programming library, it has other components already defined, which you can take from the framework and easily work on them.

So, there are many, like, you have Flask, you have Django, you have Pyramid, so these are different frameworks which are, you will often hear, from your seniors that, I am, I am, I have Django skills. So, that means that he or she is working with this framework to use Python to connect to different databases.

So, these are, Django and Pyramid are big frameworks. You can learn it over a period of time. Flask, Bottle, these are kind of smaller, micro frameworks where you can do limited things. Otherwise, you can use Python standard library support for different internet protocols. You can use package Index which has lot of other libraries.

So, it is not possible to just keep on talking on them. These are just pointers to tell you that there are lot of things going to the Python documentation of different modules and frameworks and start using them. Learning, in all these, learning in developing an application is basically starting to use it. Use of framework and keep doing it.

(Refer Slide Time 20:18)

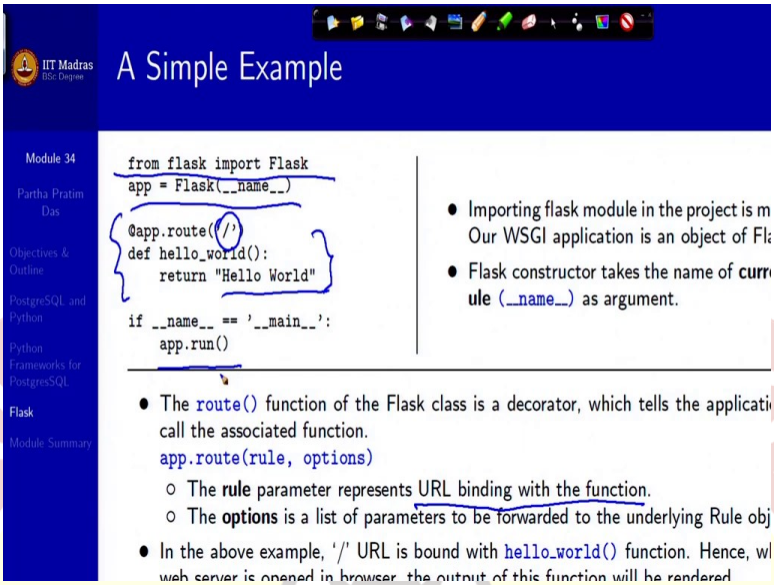


So just to give you an idea of what, I mean, what kind of facilities framework give and particularly, I have mentioned that you get certain benefits in terms of doing the front-end, let me just walk you through few features of Flask, which is a lightweight, I said a micro framework, of WSGI, which is Web Server Gateway Interface.

Why is it Web Server Gateway Interface? Because you have the web server to which you are sending the request from Python, we have discussed that earlier. Now, you are providing a different interface for that. The interface which is at the higher level, than what you will actually need to do on the web server. So, that it becomes easier for you to do the coding, do the modeling and do the testing and make it ready.

So, it, it began as a wrapper earlier, with the, of the earlier frameworks but now, it is pretty stable and pretty popular. It will also offer you suggestions of well, include this, include that dependency and so on. You may take that but, you may skip also. And, there are many extensions that are being provided by the community regularly, which gives you more and more functionality to speed up your development. Naturally, this is how you can install it.

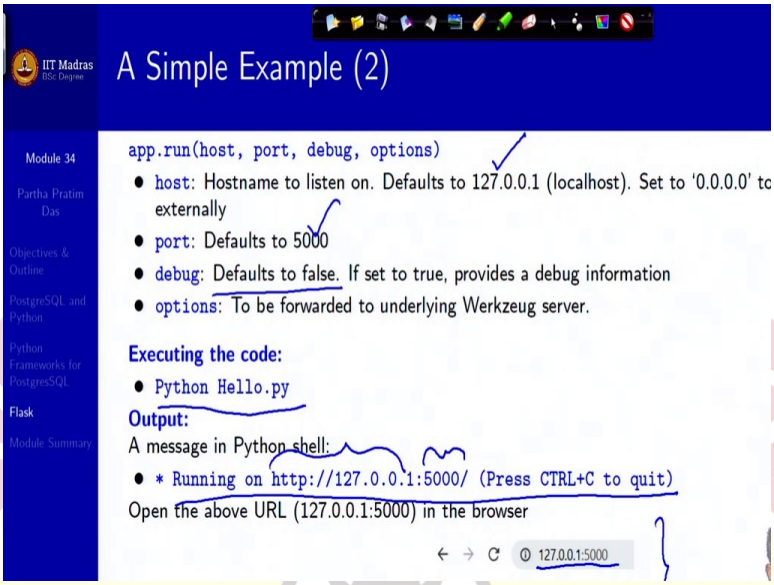
(Refer Slide Time 21:42)



So, here is a simple example to get started. So, you basically write this, at the beginning, from flask import Flask. So, you need to get the framework imported. I mean, much in the similar way that you import modules. And, then you have app a with a parameter name. Then, in the next, in the route, you basically do a binding. So, route has a rule and other options.

What the rule says is, it represents a URL binding with a function. So here, the URL is slash, forward slash. What it means? It basically means the route, homepage of your own web server. And what is the function? The function is def hello world, which just returns Hello World. So, if you do app dot run, it will run this function, method Hello World, which is bound at the route of, or home page of your web server. And you will be able to see the hello world coming on your browser. So, that is the Hello World program.

(Refer Slide Time 23:09)

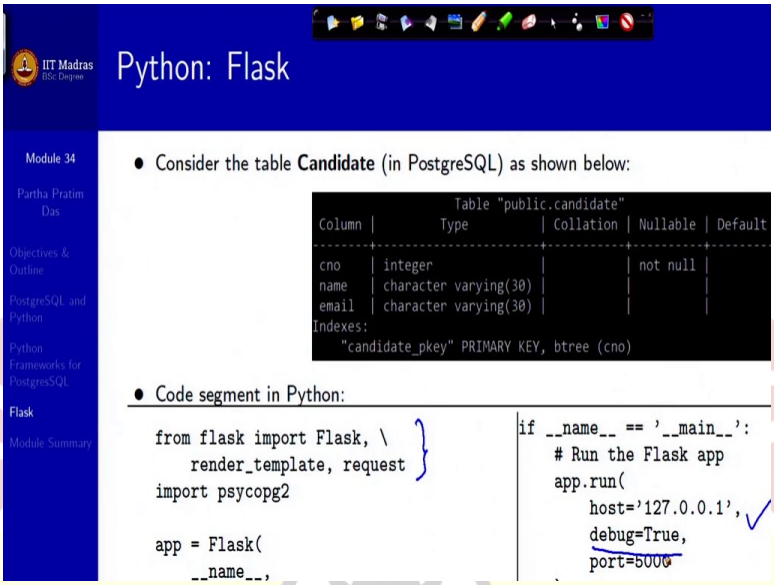


So, to build up a simple example, let us see what the app dot run might need to know. It will need to talk about the host, because you need to know where the service will be provided. The port, on which it will be there, do not really get concerned about what is the meaning of port if you do not already know. You will learn it in the course of time, anyway.

You can specify, whether you want to do debugging, and you can have other options for the web server. So, you execute this by simple, so this is a Hello.py code and you execute it simply by Python Hello.py. So, this is the message. It is the output which says running on http this, which is what you have provided as a host, colon, which is the port and you have not, you have not said debug. So, the default is false.

So, that is what, and this is the output that you get to see. You can see the host and the port are shown here, and this is what your function Hello.py is actually doing, saying Hello World. So, you can see that you did not have to, for getting this, on the browser, you did not have to do any html programming or any scripting or anything of that sort. You just wrote a simple Python code, used Flask and said, do this. Flask does that for you. That is the big advantage.

(Refer Slide Time 25:03)

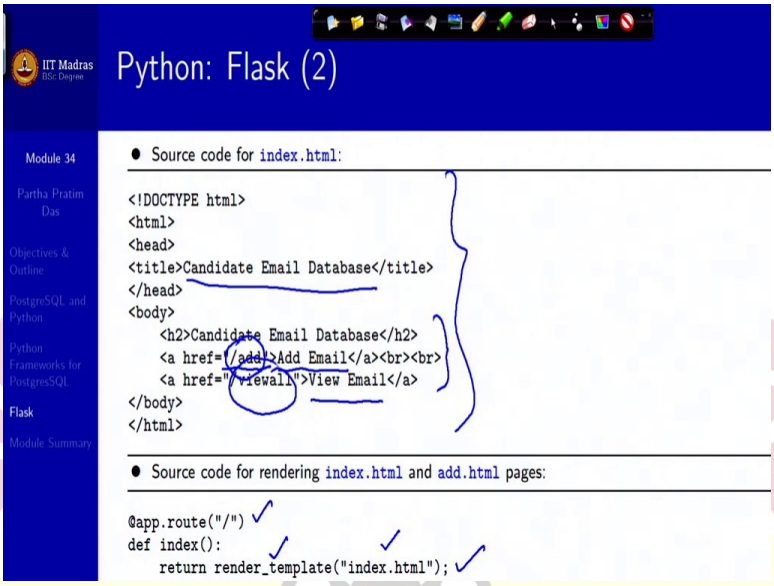


So, here is more, going deeper. You are importing, here, from import Flask, you are importing two components, very important components, render template. Render template is, we talked about that when an html code would be interpreted by the browser, the browser has a rendering engine, which decides that different html components, how will they look in the visibility.

So, to be able to do that, you need to define the rendering. You can do your own rendering but Flask gives you a rendering template. It already has a template. So, and it is often a nice one. So you can just use that template so that you (may), you just do not need to do a lot of front end programming.

So, you can say, you are specifying where the template is located and then you do this, you do the app run, exactly as you did before but you are just putting the debugging to be true, so you will be able to debug.

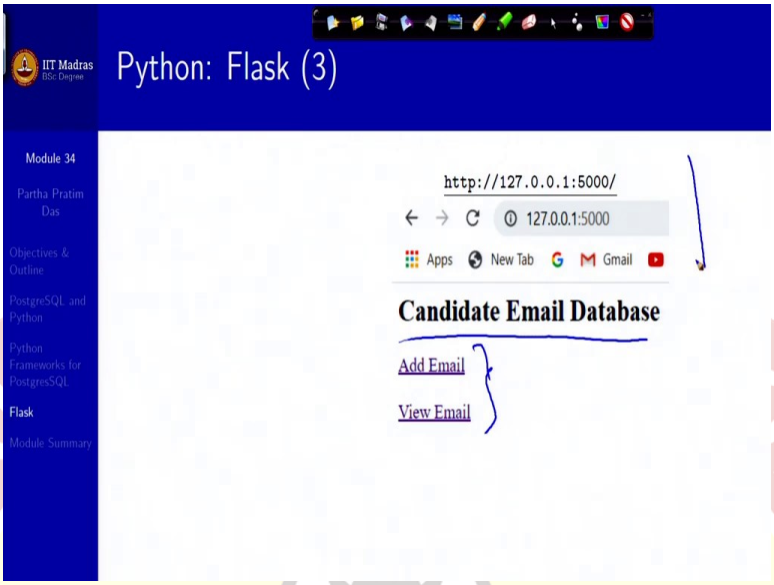
(Refer Slide Time 26:20)



Now, let us say that I have index.html, which is the beginning of everything, like a, like this kind of an html, which says it is candidate mail address is a title and you have candidate email database. And what you want, you want an Add Email link. Why link? Because you are saying href. So, you want an add mail, Add Email link which is called Add Email, and which will actually invoke add.html, so add. Similarly, you have a view email link which will invoke this.

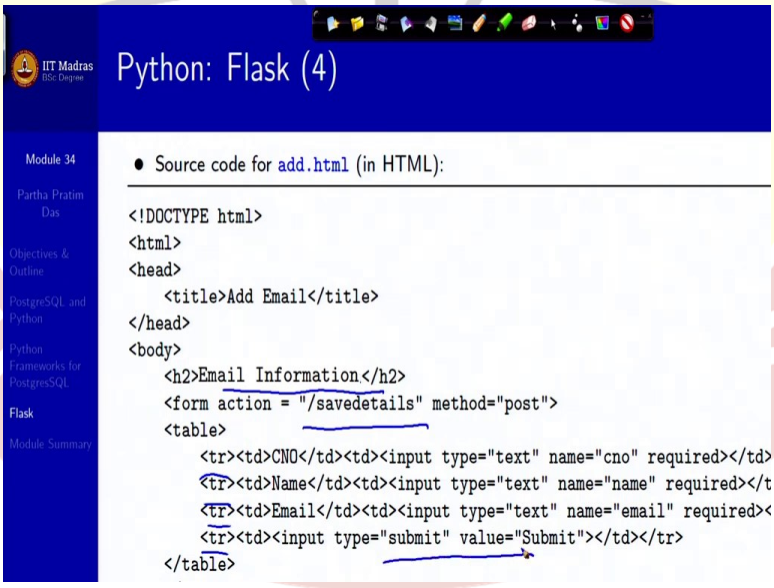
So, how do you bind it in Flask? You will say that the route I want to do index, so what you are doing is, this index function, which is by default bound at the root, is using the rendered template to render this. So, the rendered template has a nice way of showing the text. You have not specified how to show the text, you have not given any style or anything. But the rendered template will do that and show it. Similarly, you can do for the add template. So, that is how the rendered template becomes a good convenience for doing your Flask.

(Refer Slide Time 27:48)



So, once you do, you get to see this, you can see the Add Email, View Email have come as links. This is nicely shown, there is a top header, all this, and all this you have not coded for. It is coming from the rendering, default rendering in the Flask.

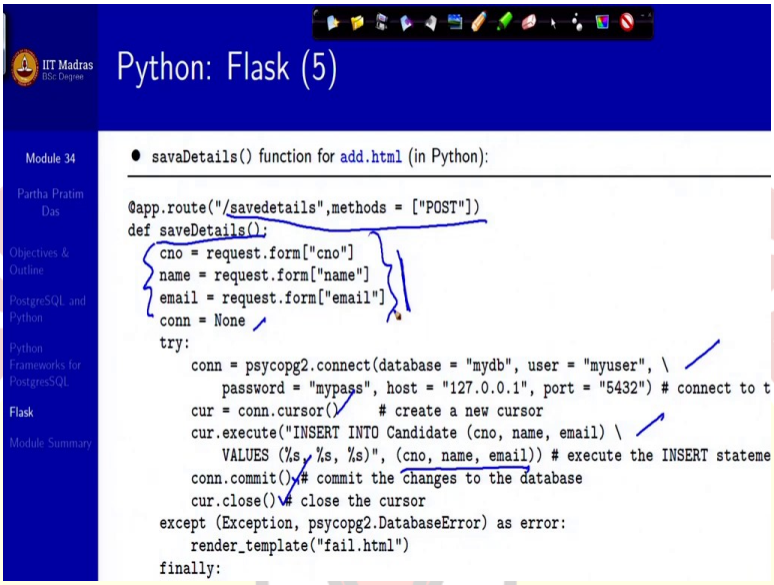
(Refer Slide Time 28:06)



This is your add.html which is, you are saying you should add email information, action is saved detail, so you will have a save details to actually keep this details and then you have specified a table. You can see tr, tr, tr, tr, which, the first three, tells you what are the different things that

you need, the boxes and the last one is a submit button. We have discussed this in the html, in the last, in the last module, I mean, module before the last.

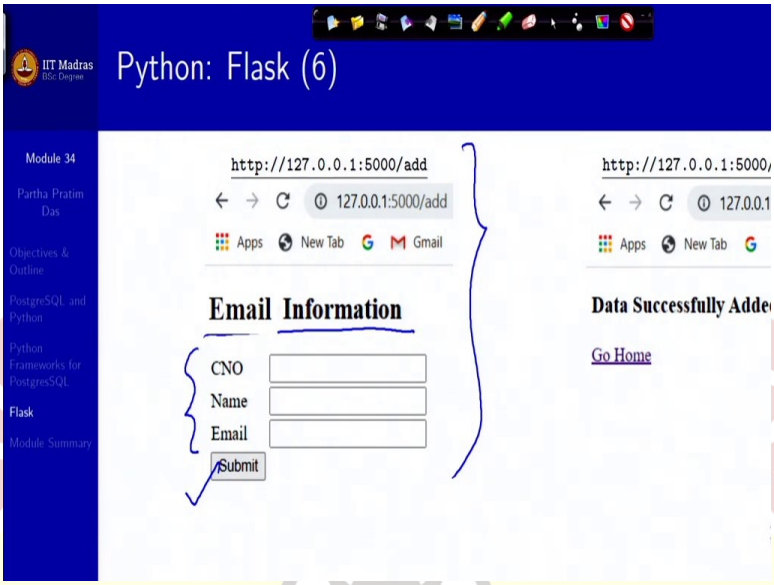
(Refer Slide Time 28:43)



So, what you do now, is this, save details, which will bound, bind with the post. Submit is, posting that information. So, what you will bind? Save details, will take the three values, that as you had specified, in the add.html and put them in the Python variables. And you do not have a connection. So, then you connect, do the cursor, you insert these three, using these, you commit, close the cursor.

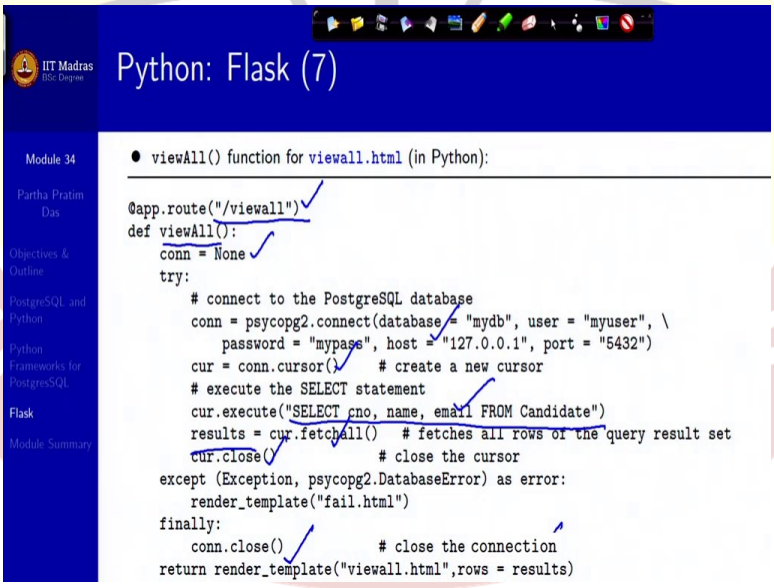
So, what will happen? It will, you have taken from the form, through here and you have put it to the database. Very simple. This is, this is all that you need. This is all that you would be needing, variants of that and so on.

(Refer Slide Time 29:40)



So, this is how it looks. It does this, there is email information, you wrote. These are the three boxes you created, this is a submit for post and once you have done, this the successful page, the success page, the, which shows you this.

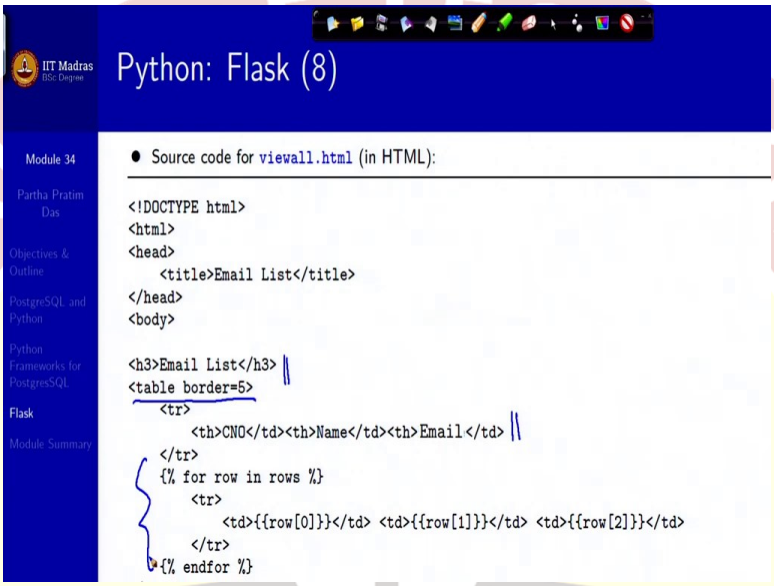
(Refer Slide Time 30:04)



So this is a view all, if you want to see all the data that has been there, so you can see that it is view-all from that, you remember, we did add email, we did view email, so the view email is bound to this method under the root. So, you have view all, connection is null, connect cursor and you execute the select statement. You are trying to see all, so there is no where clause.

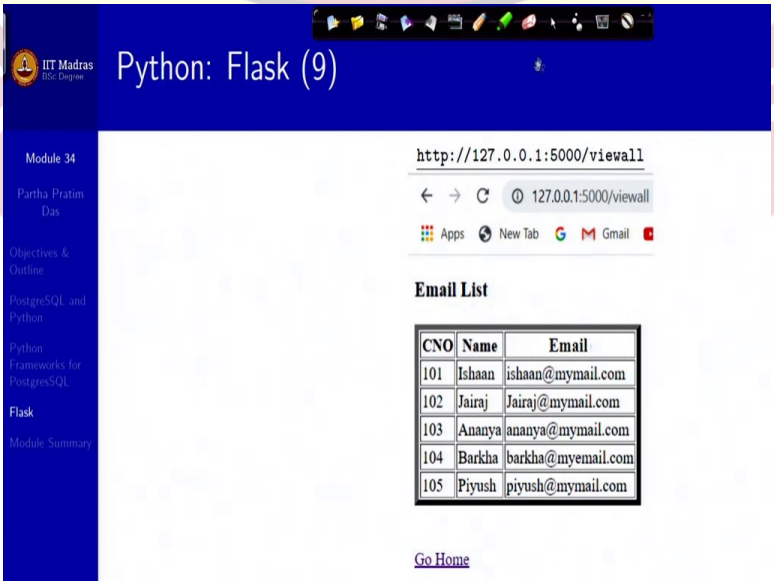
You get all, you fetch, you close the connection, and now you have the table in your results. So now, you will have to show it in the page. So, what you return, you return rendered template which will return it, as a part of view all html, where rows is equal to results. So it will, so this is where the data is given, it will automatically render and show you.

(Refer Side Time 31:18)



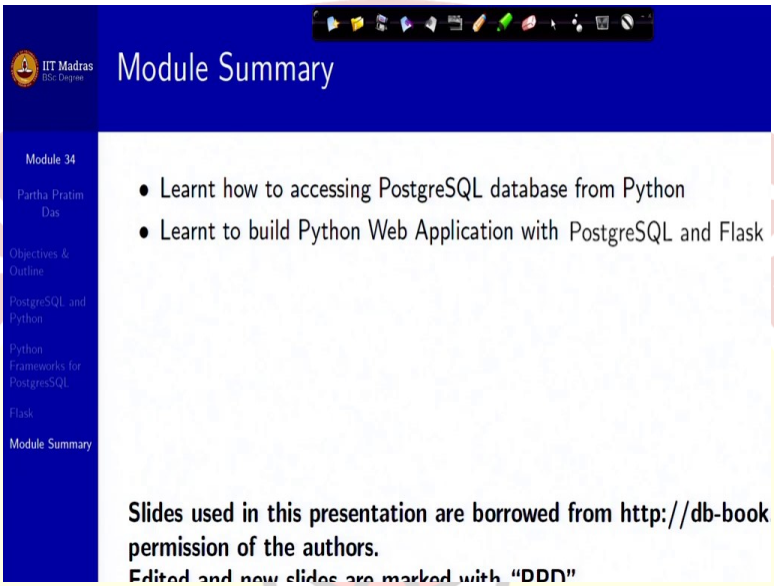
So, this is view html structure. You have the email list. You have that border type. This is your header, and the rendering will populate this data as they have come in the results into this table and will take you to that rendered page.

(Refer Slide Time 31:43)



So, as you do this, this is what you are, you get to see, this is your rendered list. This is your rendered list, this is the Go Home link, this is the header that you have written in the view all dot html.

(Refer Slide Time 32:05)



So using Flask, you can very easily develop a total integrated web application using Python as a middle tier language and PostgreSQL as a database. I hope you start doing these developments very soon. Thank you very much for your attention. See you in the next module.